
Sampling Coding Test Guide

Concepts, formulas, Python patterns, and 10 solved problems



What this guide covers

Impulse-train sampling; spectrum replication; Nyquist rate; aliasing; safe downsampling and anti-alias filters; ideal sinc, zero-order hold, and linear reconstruction; DTFT/DFT frequency mapping; reusable NumPy/SciPy code; ten coding-test problems with complete solutions.

Based on the supplied 43-page class lecture on Sampling

Designed for fast revision and direct coding-test use

Contents

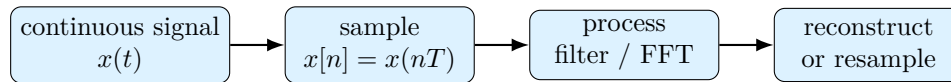
1	The Big Picture	1
2	Core Theory from the Lecture	1
2.1	Impulse-train sampling	1
2.2	Nyquist-Shannon sampling theorem	1
2.3	Aliasing: frequency folding	2
2.4	Downsampling and anti-alias filtering	2
2.5	Signal reconstruction	2
2.6	DTFT and DFT connection	3
3	Formula and Decision Cheat Sheet	3
4	Reusable Python Toolkit	3
4.1	Imports, time axis, and signal generation	4
4.2	One-sided amplitude spectrum	4
4.3	Aliased-frequency predictor	4
4.4	Naive versus safe downsampling	4
4.5	ZOH, linear, and finite-sinc reconstruction	5
4.6	Five plotting and implementation traps	5
5	Ten Solved Coding Problems	5
5.1	Problem 1: Generate and sample a two-tone signal	5
5.2	Problem 2: Demonstrate aliasing experimentally	6
5.3	Problem 3: Write a general alias calculator	6
5.4	Problem 4: Read physical frequencies from a DFT	6
5.5	Problem 5: Naive downsampling versus protected downsampling	7
5.6	Problem 6: Design the anti-alias FIR yourself	8
5.7	Problem 7: Compare ZOH, linear, and sinc reconstruction	8
5.8	Problem 8: Choose a sampling rate automatically	9
5.9	Problem 9: Downsample a 2-D image with anti-aliasing	9
5.10	Problem 10: Capstone – resample audio from 48 kHz to 16 kHz	10
6	Fast Exam Strategy	10
7	Last-Minute Checklist	11

1 The Big Picture

A continuous-time signal is sampled every T seconds:

$$x[n] = x(nT), \quad f_s = \frac{1}{T}, \quad \omega_s = 2\pi f_s = \frac{2\pi}{T}.$$

The samples determine the original signal uniquely only when the signal is band-limited and the sampling rate is high enough. In code, most mistakes come from confusing continuous frequency f in hertz, digital frequency Ω in radians/sample, and FFT bin index k .



Coding rule that prevents many bugs

Use `endpoint=False` when generating a sampled time grid. The samples in a duration D are normally at $0, T, 2T, \dots, D - T$, so the count is $N = Df_s$ when this is an integer.

2 Core Theory from the Lecture

2.1 Impulse-train sampling

Sampling can be represented as multiplication by an impulse train:

$$p(t) = \sum_{n=-\infty}^{\infty} \delta(t - nT), \quad x_p(t) = x(t)p(t) = \sum_{n=-\infty}^{\infty} x(nT)\delta(t - nT).$$

The impulse train has Fourier transform

$$P(j\omega) = \frac{2\pi}{T} \sum_{k=-\infty}^{\infty} \delta(\omega - k\omega_s).$$

Since multiplication in time becomes convolution in frequency,

$$X_p(j\omega) = \frac{1}{T} \sum_{k=-\infty}^{\infty} X(j(\omega - k\omega_s)).$$

Therefore sampling creates shifted copies of the original spectrum, centered at $0, \pm\omega_s, \pm2\omega_s, \dots$, each scaled by $1/T$.

2.2 Nyquist-Shannon sampling theorem

If $X(j\omega) = 0$ for $|\omega| > \omega_M$, exact recovery is possible when

$$\boxed{\omega_s > 2\omega_M} \quad \Longleftrightarrow \quad \boxed{f_s > 2f_M}.$$

The value $2f_M$ is the **Nyquist rate**. The value $f_s/2$ is the **Nyquist frequency** of a sampler. These names are often confused.

Term	Meaning
Sampling rate f_s	Number of samples taken per second, in Hz.
Nyquist rate $2f_M$	Minimum theoretical sampling rate for a signal whose highest frequency is f_M .
Nyquist frequency $f_s/2$	Highest unaliased frequency representable at a particular f_s .
Oversampling	$f_s > 2f_M$; spectral copies are separated by a guard band.
Critical sampling	$f_s = 2f_M$; fragile in practice because real filters need a transition band.
Undersampling	$f_s < 2f_M$; copies overlap and aliasing occurs.

2.3 Aliasing: frequency folding

Two continuous sinusoids whose frequencies differ by an integer multiple of f_s produce the same discrete-time frequency:

$$e^{j2\pi(f+mf_s)n/f_s} = e^{j2\pi fn/f_s}, \quad m \in \mathbb{Z}.$$

To fold a frequency into the signed Nyquist interval $[-f_s/2, f_s/2)$, use

$$f_{\text{signed}} = ((f + f_s/2) \bmod f_s) - f_s/2.$$

For a real cosine, the visible nonnegative alias is

$$f_{\text{alias}} = |f_{\text{signed}}|.$$

Example: a 900 Hz cosine sampled at 1000 Hz appears at $|900 - 1000| = 100$ Hz.

Aliasing is irreversible after sampling

Once two spectral components have folded onto the same discrete frequency, a digital filter cannot determine which original component produced them. The anti-alias filter must act *before* the rate reduction or A/D sampling.

2.4 Downsampling and anti-alias filtering

Downsampling by an integer factor M keeps every M -th sample:

$$y[n] = x[nM], \quad f'_s = \frac{f_s}{M}.$$

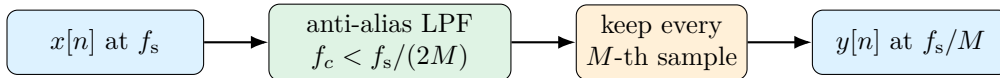
Its DTFT is

$$Y(e^{j\Omega}) = \frac{1}{M} \sum_{r=0}^{M-1} X(e^{j(\Omega+2\pi r)/M}).$$

The sum shows that M compressed spectral copies are added. To prevent overlap, low-pass filter before downsampling:

$$f_c < \frac{f'_s}{2} = \frac{f_s}{2M}.$$

In normalized digital frequency, the ideal cutoff is $\Omega_c \leq \pi/M$. A practical filter needs a transition band, so choose f_c slightly below $f_s/(2M)$.



2.5 Signal reconstruction

For ideal reconstruction, keep the baseband copy with an ideal low-pass filter:

$$H(j\omega) = \begin{cases} T, & |\omega| < \omega_s/2, \\ 0, & \text{otherwise.} \end{cases}$$

Using the normalized definition $\text{sinc}(u) = \sin(\pi u)/(\pi u)$,

$$h(t) = \text{sinc}(t/T), \quad x(t) = \sum_{n=-\infty}^{\infty} x[n] \text{sinc}\left(\frac{t-nT}{T}\right).$$

The ideal sinc is non-causal and infinite in duration, so practical systems approximate it.

Method	Time-domain behavior	Frequency-domain behavior
Ideal sinc	Exact for a Nyquist-sampled band-limited signal	Perfect rectangular baseband selection; unrealizable.
Zero-order hold	Holds each sample until the next; staircase	$H_0(j\omega) = T e^{-j\omega T/2} \text{sinc}\left(\frac{\omega T}{2\pi}\right)$; pass-band droop.
Linear interpolation	Joins adjacent samples with straight lines	$H_1(j\omega) = T \text{sinc}^2\left(\frac{\omega T}{2\pi}\right)$; faster high-frequency rolloff than ZOH.

2.6 DTFT and DFT connection

The DTFT of a sample sequence is periodic with period 2π :

$$X(e^{j\Omega}) = \sum_{n=-\infty}^{\infty} x[n] e^{-j\Omega n}.$$

Physical and digital frequencies are related by

$$\boxed{\Omega = 2\pi \frac{f}{f_s}}, \quad \boxed{f = \frac{\Omega}{2\pi} f_s}.$$

An N -point DFT samples one DTFT period at $\Omega_k = 2\pi k/N$:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}, \quad f_k = \frac{k f_s}{N}, \quad \Delta f = \frac{f_s}{N}.$$

For an unshifted FFT, bins above $N/2$ represent wrapped negative frequencies. Use `np.fft.fftfreq`; for real signals, prefer `np.fft.rfftfreq`.

For a periodic signal sampled exactly N times per period, the DFT bins correspond to its harmonic Fourier-series coefficients; under the lecture's convention, $X[k] = N a_k$ for harmonics inside the Nyquist band.

3 Formula and Decision Cheat Sheet

Question	Formula / decision
Sampling period	$T = 1/f_s$
Angular sampling frequency	$\omega_s = 2\pi f_s = 2\pi/T$
Nyquist condition	$f_s > 2f_M$; use margin in practical systems
New rate after downsampling by M	$f'_s = f_s/M$
Anti-alias cutoff before downsampling	$f_c < f_s/(2M) = f'_s/2$
Digital frequency	$\Omega = 2\pi f/f_s$ rad/sample
FFT bin spacing	$\Delta f = f_s/N = 1/D$ for duration $D = N/f_s$
Physical frequency of bin k	$f_k = k f_s/N$ before negative-frequency wrapping
Cosine alias in $[0, f_s/2]$	$ ((f + f_s/2) \bmod f_s) - f_s/2 $
Ideal interpolation	$x(t) = \sum_n x[n] \text{sinc}((t - nT)/T)$
ZOH in code	Repeat values or use previous-sample indexing
Linear interpolation in code	<code>np.interp(t_fine, t_s, x_s)</code>
Safe rational resampling	<code>scipy.signal.resample_poly(x, L, M)</code>
L/M	

4 Reusable Python Toolkit

4.1 Imports, time axis, and signal generation

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy import signal
4
5 fs = 1000.0
6 duration = 1.0
7 t = np.arange(0, duration, 1/fs) # endpoint excluded
8 x = np.cos(2*np.pi*80*t) + 0.5*np.cos(2*np.pi*200*t)
9 print(len(x), len(x)/fs)         # sample count, actual duration

```

4.2 One-sided amplitude spectrum

For a real signal, this helper returns a convenient one-sided amplitude estimate. It doubles non-DC/non-Nyquist bins to account for the omitted negative-frequency half.

```

1 def one_sided_spectrum(x, fs):
2     x = np.asarray(x)
3     N = len(x)
4     X = np.fft.rfft(x)
5     f = np.fft.rfftfreq(N, d=1/fs)
6     A = np.abs(X) / N
7     if N > 1:
8         A[1:-1] *= 2           # correct for even N
9         if N % 2 == 1:
10            A[-1] *= 2          # last bin is not Nyquist for odd N
11     return f, A
12
13 f, A = one_sided_spectrum(x, fs)
14 plt.plot(f, A)
15 plt.xlim(0, fs/2)
16 plt.xlabel("Frequency (Hz)")
17 plt.ylabel("Amplitude")
18 plt.grid(True)

```

4.3 Aliased-frequency predictor

```

1 def signed_alias(f, fs):
2     return (f + fs/2) % fs - fs/2
3
4 def cosine_alias(f, fs):
5     return abs(signed_alias(f, fs))
6
7 for f0 in [100, 510, 750, 900, 1300]:
8     print(f0, "->", cosine_alias(f0, 1000), "Hz")

```

4.4 Naive versus safe downsampling

```

1 M = 4
2 y_bad = x[::M]                # no anti-alias protection
3 fs_new = fs / M
4 y_good = signal.resample_poly(x, up=1, down=M)

```

`resample_poly` applies a low-pass filter internally, then changes the rate. For a coding test, it is the safest short implementation. If asked to design the anti-alias filter yourself, use `firwin` and filter before slicing.

```

1 M = 4

```

```

2 fs_new = fs / M
3 cutoff = 0.9 * (fs_new / 2)           # transition margin
4 b = signal.firwin(numtaps=129, cutoff=cutoff, fs=fs)
5 x_filt = signal.lfilter(b, [1.0], x)
6 delay = (len(b) - 1) // 2             # FIR group delay in samples
7 y_manual = x_filt[delay::M]

```

4.5 ZOH, linear, and finite-sinc reconstruction

```

1 def reconstruct_zoh(t_samples, x_samples, t_fine):
2     idx = np.searchsorted(t_samples, t_fine, side="right") - 1
3     idx = np.clip(idx, 0, len(x_samples)-1)
4     return x_samples[idx]
5
6 def reconstruct_linear(t_samples, x_samples, t_fine):
7     return np.interp(t_fine, t_samples, x_samples)
8
9 def reconstruct_sinc(t_samples, x_samples, t_fine, fs):
10    # np.sinc(u) = sin(pi*u)/(pi*u)
11    kernel = np.sinc(fs * (t_fine[:, None] - t_samples[None, :]))
12    return kernel @ x_samples

```

4.6 Five plotting and implementation traps

1. **Wrong time grid:** `np.linspace(0,D,int(D*fs))` includes both ends by default and therefore does not have spacing exactly $1/f_s$. Use `np.arange` or `endpoint=False`.
2. **Wrong FFT axis:** never label bin index k directly as Hz. Use `rfftfreq` or `fftfreq`.
3. **Filtering after slicing:** that cannot undo aliasing. Filter first, then downsample.
4. **FFT leakage mistaken for aliasing:** leakage is energy spread caused by a finite observation/window; aliasing is frequency folding caused by sampling.
5. **Ignoring filter delay:** a causal linear-phase FIR delays the signal by $(L-1)/2$ samples. Compensate when comparing waveforms, or use zero-phase filtering for offline analysis.

5 Ten Solved Coding Problems

Each problem is written in the style of a short coding test. Try the task first, then check the solution and expected observation.

5.1 Problem 1: Generate and sample a two-tone signal

Task

Generate $x(t) = \cos(2\pi 50t) + 0.4 \sin(2\pi 120t)$ for 0.1 s. Sample it at 1000 Hz. Plot the sampled values with a stem plot and print the number of samples. State whether Nyquist is satisfied.

Solution and reasoning

The highest component is 120 Hz, so the Nyquist rate is 240 Hz. Since $1000 > 240$, sampling is safe.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 fs, D = 1000.0, 0.1
5 t = np.arange(0, D, 1/fs)

```

```

6 x = np.cos(2*np.pi*50*t) + 0.4*np.sin(2*np.pi*120*t)
7
8 print("samples =", len(x))           # 100
9 print("Nyquist satisfied:", fs > 2*120)
10 plt.stem(t, x, basefmt=" ")
11 plt.xlabel("Time (s)"); plt.ylabel("x[n]"); plt.grid(True)
12 plt.show()

```

5.2 Problem 2: Demonstrate aliasing experimentally

Task

Sample a 120 Hz cosine at (a) 1000 Hz and (b) 180 Hz, each for 1 s. Use the FFT to find the dominant frequency in both sampled signals. Explain the second result.

Expected result

At 1000 Hz the peak is 120 Hz. At 180 Hz, 120 Hz folds to $|120 - 180| = 60$ Hz because the Nyquist frequency is only 90 Hz.

```

1 def dominant_frequency(x, fs):
2     f = np.fft.rfftfreq(len(x), d=1/fs)
3     A = np.abs(np.fft.rfft(x))
4     return f[np.argmax(A[1:])] + 1 # ignore DC
5
6 for fs in [1000.0, 180.0]:
7     t = np.arange(0, 1.0, 1/fs)
8     x = np.cos(2*np.pi*120*t)
9     print(fs, dominant_frequency(x, fs))

```

5.3 Problem 3: Write a general alias calculator

Task

Write a function that maps any real sinusoidal frequency to the signed Nyquist interval $[-f_s/2, f_s/2)$. Then report both signed and cosine aliases for [100, 510, 750, 900, 1300] Hz when $f_s = 1000$ Hz.

```

1 def signed_alias(f, fs):
2     return (np.asarray(f) + fs/2) % fs - fs/2
3
4 freqs = np.array([100, 510, 750, 900, 1300])
5 s = signed_alias(freqs, 1000)
6 print("signed:", s)           # [100, -490, -250, -100, 300]
7 print("cosine:", np.abs(s))   # [100, 490, 250, 100, 300]

```

Key idea

Modulo by f_s first expresses the equivalence $f \sim f + m f_s$. The shift by $f_s/2$ centers the result around zero. Taking the absolute value is valid for a real cosine because $\cos(-\theta) = \cos(\theta)$.

5.4 Problem 4: Read physical frequencies from a DFT

Task

At $f_s = 1000$ Hz, create one second of $x[n] = 1.2 \cos(2\pi 80n/f_s) + 0.5 \cos(2\pi 200n/f_s)$. Use an RFFT to estimate the two frequencies and amplitudes. Also print Δf .

```

1 fs, D = 1000.0, 1.0
2 t = np.arange(0, D, 1/fs)
3 x = 1.2*np.cos(2*np.pi*80*t) + 0.5*np.cos(2*np.pi*200*t)
4
5 N = len(x)
6 f = np.fft.rfftfreq(N, d=1/fs)
7 A = 2*np.abs(np.fft.rfft(x))/N
8 A[0] /= 2
9 if N % 2 == 0:
10     A[-1] /= 2
11
12 peak_idx = np.argsort(A[1:])[-2:] + 1
13 peak_idx = peak_idx[np.argsort(f[peak_idx])]
14 print("df =", fs/N, "Hz") # 1 Hz
15 print(list(zip(f[peak_idx], A[peak_idx])))
16 # approximately [(80, 1.2), (200, 0.5)]

```

Why the result is clean

The observation duration is one second, so $\Delta f = 1$ Hz. Both sinusoids complete an integer number of cycles and land exactly on DFT bins, so there is no leakage in exact arithmetic.

5.5 Problem 5: Naive downsampling versus protected downsampling

Task

Create a signal sampled at 4000 Hz containing 200 Hz and 900 Hz cosines. Downsample by $M = 4$ (a) by slicing and (b) using polyphase resampling. Plot both spectra and explain the peaks.

```

1 from scipy import signal
2
3 fs, D, M = 4000.0, 2.0, 4
4 t = np.arange(0, D, 1/fs)
5 x = np.cos(2*np.pi*200*t) + 0.8*np.cos(2*np.pi*900*t)
6
7 y_bad = x[::M]
8 y_good = signal.resample_poly(x, up=1, down=M)
9 fs2 = fs/M # 1000 Hz, Nyquist = 500 Hz
10
11 fig, ax = plt.subplots(2, 1, sharex=True)
12 for a, y, title in zip(ax, [y_bad, y_good],
13                          ["Naive", "Anti-aliased"]):
14     f, A = one_sided_spectrum(y, fs2)
15     a.plot(f, A); a.set_title(title); a.grid(True)
16 ax[-1].set_xlabel("Frequency (Hz)")
17 plt.tight_layout(); plt.show()

```

Expected result

The naive output has peaks near 200 Hz and 100 Hz because 900 Hz aliases to $|900 - 1000| = 100$ Hz. The protected output retains the 200 Hz tone and strongly suppresses the original 900 Hz component before rate reduction.

5.6 Problem 6: Design the anti-alias FIR yourself

Task

Convert an 8000 Hz signal to 2000 Hz by downsampling by four. It contains 300 Hz and 1500 Hz tones. Design a 129-tap FIR with a 900 Hz cutoff, filter before slicing, compensate the FIR delay, and verify the output spectrum.

```

1 fs, D, M = 8000.0, 2.0, 4
2 t = np.arange(0, D, 1/fs)
3 x = np.cos(2*np.pi*300*t) + 0.7*np.cos(2*np.pi*1500*t)
4
5 numtaps, cutoff = 129, 900.0
6 b = signal.firwin(numtaps, cutoff=cutoff, fs=fs)
7 xf = signal.lfilter(b, [1.0], x)
8 delay = (numtaps - 1)//2
9
10 # Remove startup delay, then keep every fourth sample.
11 y = xf[delay::M]
12 fs2 = fs/M
13 f, A = one_sided_spectrum(y, fs2)
14 print("dominant =", f[np.argmax(A[1:])+1], "Hz")
15 plt.plot(f, A); plt.xlim(0, fs2/2); plt.grid(True); plt.show()

```

Expected result

The new Nyquist frequency is 1000 Hz. The cutoff at 900 Hz leaves transition room. The 300 Hz component remains; the 1500 Hz component is heavily attenuated and therefore does not fold to 500 Hz.

5.7 Problem 7: Compare ZOH, linear, and sinc reconstruction

Task

Sample a 5 Hz sine wave at 16 Hz for two seconds. Reconstruct it on a 2000 Hz plotting grid using ZOH, linear interpolation, and finite-record sinc interpolation. Compute RMSE only on the central interval [0.25, 1.75] s to reduce sinc edge effects.

```

1 fs, D, f0 = 16.0, 2.0, 5.0
2 ts = np.arange(0, D, 1/fs)
3 xs = np.sin(2*np.pi*f0*ts)
4 tf = np.arange(0, D, 1/2000)
5 truth = np.sin(2*np.pi*f0*tf)
6
7 zoh = reconstruct_zoh(ts, xs, tf)
8 linear = reconstruct_linear(ts, xs, tf)
9 sinc_rec = reconstruct_sinc(ts, xs, tf, fs)
10
11 mask = (tf >= 0.25) & (tf <= 1.75)
12 for name, y in [("ZOH", zoh), ("Linear", linear), ("Sinc", sinc_rec)]:
13     rmse = np.sqrt(np.mean((y[mask] - truth[mask])**2))
14     print(name, rmse)
15
16 plt.plot(tf, truth, "k", label="true")
17 plt.plot(tf, zoh, label="ZOH", alpha=.8)
18 plt.plot(tf, linear, label="linear")
19 plt.plot(tf, sinc_rec, label="finite sinc")
20 plt.scatter(ts, xs, c="k", s=12, zorder=5, label="samples")
21 plt.xlim(.25, .75); plt.legend(); plt.grid(True); plt.show()

```

Interpretation

Nyquist holds because $16 > 2(5)$. ZOH is staircase-shaped, linear is smoother, and ideal infinite sinc would reproduce the band-limited sinusoid exactly. The code uses a finite record, so its sinc sum has boundary error; central-interval RMSE is the fairer comparison.

5.8 Problem 8: Choose a sampling rate automatically

Task

Given sinusoidal components at 60, 180, and 310 Hz, write a function that returns (a) the theoretical Nyquist rate and (b) the smallest allowed rate from [500, 600, 800, 1000] Hz with at least a 20% guard margin above Nyquist. Show what the 310 Hz tone becomes at 600 Hz.

```

1 def choose_fs(components, allowed, margin=0.20):
2     nyquist_rate = 2*max(abs(np.asarray(components)))
3     required = (1 + margin)*nyquist_rate
4     valid = [fs for fs in allowed if fs >= required]
5     if not valid:
6         raise ValueError("No allowed sampling rate is high enough")
7     return nyquist_rate, min(valid)
8
9 components = [60, 180, 310]
10 nyq, chosen = choose_fs(components, [500, 600, 800, 1000])
11 print(nyq, chosen)           # 620 Hz, 800 Hz
12 print(cosine_alias(310, 600)) # 290 Hz

```

Reasoning

The theoretical rate is $2(310) = 620$ Hz. With 20% margin, the requirement is 744 Hz, so 800 Hz is the first allowed choice. At 600 Hz, the Nyquist frequency is 300 Hz and 310 Hz folds to 290 Hz.

5.9 Problem 9: Downsample a 2-D image with anti-aliasing

Task

Create a high-frequency checkerboard, reduce each dimension by four using (a) direct slicing and (b) Gaussian prefiling followed by slicing. Display the three images. Explain why false large-scale patterns can appear in the naive result.

```

1 from scipy.ndimage import gaussian_filter
2
3 N, M = 256, 4
4 r, c = np.indices((N, N))
5 img = ((r//3 + c//3) % 2).astype(float) # fine checkerboard
6
7 naive = img[:, ::M, ::M]
8 blurred = gaussian_filter(img, sigma=1.2)
9 safe = blurred[:, ::M, ::M]
10
11 fig, ax = plt.subplots(1, 3, figsize=(9, 3))
12 for a, im, title in zip(ax, [img, naive, safe],
13     ["Original", "Naive /4", "Blur then /4"]):
14     a.imshow(im, cmap="gray", vmin=0, vmax=1, interpolation="nearest")
15     a.set_title(title); a.axis("off")
16 plt.tight_layout(); plt.show()

```

Interpretation

Downsampling an image is sampling in two spatial dimensions. Fine checkerboard frequencies above the new horizontal or vertical Nyquist limits fold into lower spatial frequencies, producing moire patterns. Blurring is a practical 2-D low-pass anti-alias filter.

5.10 Problem 10: Capstone – resample audio from 48 kHz to 16 kHz

Task

Create a synthetic audio signal at 48 kHz containing 1 kHz, 7 kHz, and 12 kHz tones. Convert it to 16 kHz by (a) taking every third sample and (b) using `resample_poly`. Write a helper that prints the three strongest spectral peaks of each result and explain the difference.

```

1  fs, D = 48000.0, 1.0
2  t = np.arange(0, D, 1/fs)
3  x = (np.cos(2*np.pi*1000*t)
4       + 0.7*np.cos(2*np.pi*7000*t)
5       + 0.8*np.cos(2*np.pi*12000*t))
6
7  M = 3
8  y_bad = x[:,M]
9  y_good = signal.resample_poly(x, up=1, down=M)
10 fs2 = fs/M                                # 16 kHz; Nyquist = 8 kHz
11
12 def strongest_peaks(x, fs, count=3):
13     f, A = one_sided_spectrum(x, fs)
14     peaks, _ = signal.find_peaks(A)
15     order = peaks[np.argsort(A[peaks])[-count:]][::-1]
16     return list(zip(np.round(f[order], 1), np.round(A[order], 3)))
17
18 print("naive:", strongest_peaks(y_bad, fs2))
19 print("safe :", strongest_peaks(y_good, fs2))

```

Expected result

The new Nyquist frequency is 8 kHz. The valid 1 kHz and 7 kHz components remain. In the naive result, 12 kHz folds to $|12000 - 16000| = 4$ kHz. Polyphase resampling suppresses frequencies above 8 kHz before decimation, so the false 4 kHz peak is strongly reduced.

6 Fast Exam Strategy

For almost every coding question, follow this order:

1. **Identify rates:** original f_s , factor M or target rate, and new Nyquist frequency.
2. **Predict before coding:** list which components survive and calculate aliases for those that do not.
3. **Build the time axis correctly:** `np.arange(0,D,1/fs)`.
4. **Perform the requested operation:** direct sampling, filter-then-slice, or `resample_poly`.
5. **Verify in frequency:** use `rfft` with `rfftfreq` and label axes in hertz.
6. **Explain one sentence of physics:** “frequencies above the new Nyquist limit fold unless removed before downsampling.”

Three sentences worth memorizing

Sampling repeats the continuous spectrum every f_s Hz. Aliasing occurs when repeated copies overlap. Safe downsampling therefore requires low-pass filtering to below the new Nyquist frequency before keeping fewer samples.

7 Last-Minute Checklist

- I can distinguish Nyquist rate $2f_M$ from Nyquist frequency $f_s/2$.
- I can calculate an alias using modulo/folding.
- I know that slicing alone is not safe decimation.
- I can choose an anti-alias cutoff below $f_s/(2M)$.
- I can generate a correct sample grid without duplicating the endpoint.
- I can map FFT bins to hertz and state $\Delta f = f_s/N$.
- I can explain ZOH, linear, and sinc reconstruction.
- I can use `resample_poly` and also implement manual FIR-then-slice decimation.
- I can separate spectral leakage from aliasing.
- I always predict the expected spectrum before trusting a plot.